



PRESENTATION

ARQUITECTURA DE BASE DE DATOS

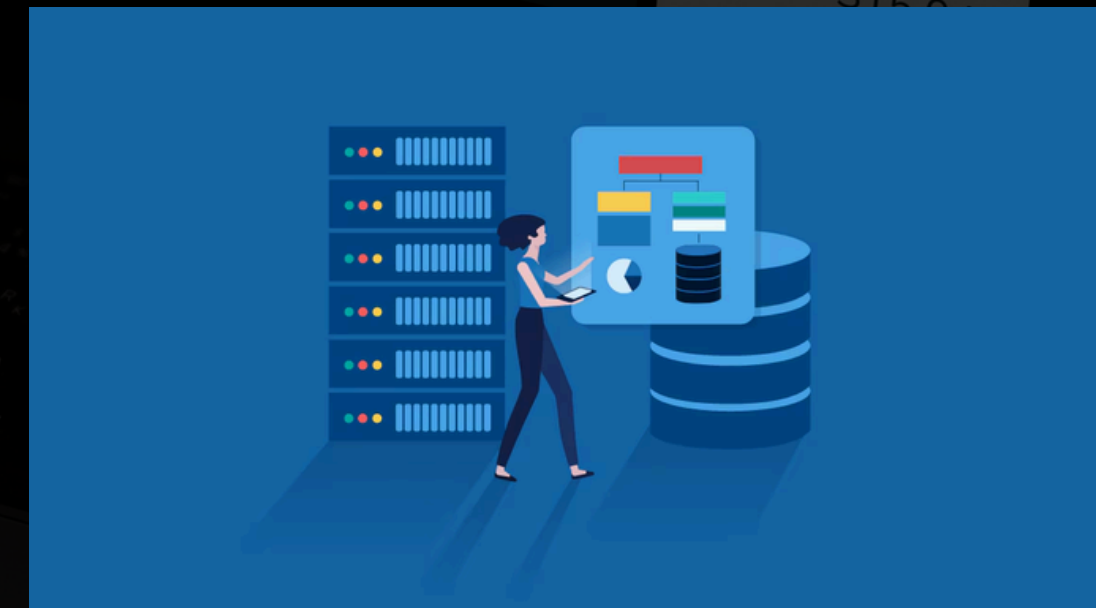


TEMA :ANALISIS PARA EL SISTEMA PARA
EL SISTEMA DE GESTION ACADEMICO

CURSO:BASE DE DATOS II

DOCENTE:ING.FERNANDEZ BEJARANO
RAUL ENRIQUE

ESTUDIANTE:ROSALES CRISPIN
ARISTOTELES ONASSIS



CICLO V - JUNIN,HUANCAYO6
2026

INDICE

PAG 3

ARQUITECTURA CENTRALIZADA

un único nodo central

PAG 4

ARQUITECTURA CLIENTE - SERVIDOR

diseño en el que las funciones
de procesamiento de datos

PAG 5

ARQUITECTUR DISTRIBUIDA

Modelo de software donde los
componentes residen en múltiples
nodos interconectados

PAG 6

CUADRO COMPARATIVO

Analisis comparativo de
las diversas arquitecturas

ARQUITECTURA CENTRALIZADA

Que es ?

Es un modelo donde un único nodo central (servidor) tiene el control absoluto. Este centro recibe todas las peticiones, procesa los datos y envía las respuestas a los "nodos periféricos" (computadoras o usuarios), que no tienen autonomía y dependen totalmente del centro para funcionar.



ARQUITECTURA CLIENTE - SERVIDOR

QUE ES?

Es un modelo de diseño en el que las funciones de procesamiento de datos se dividen entre dos entidades principales: el cliente, que solicita servicios o recursos y el servidor, que los proporciona



ARQUITECTURA DISTRIBUIDA

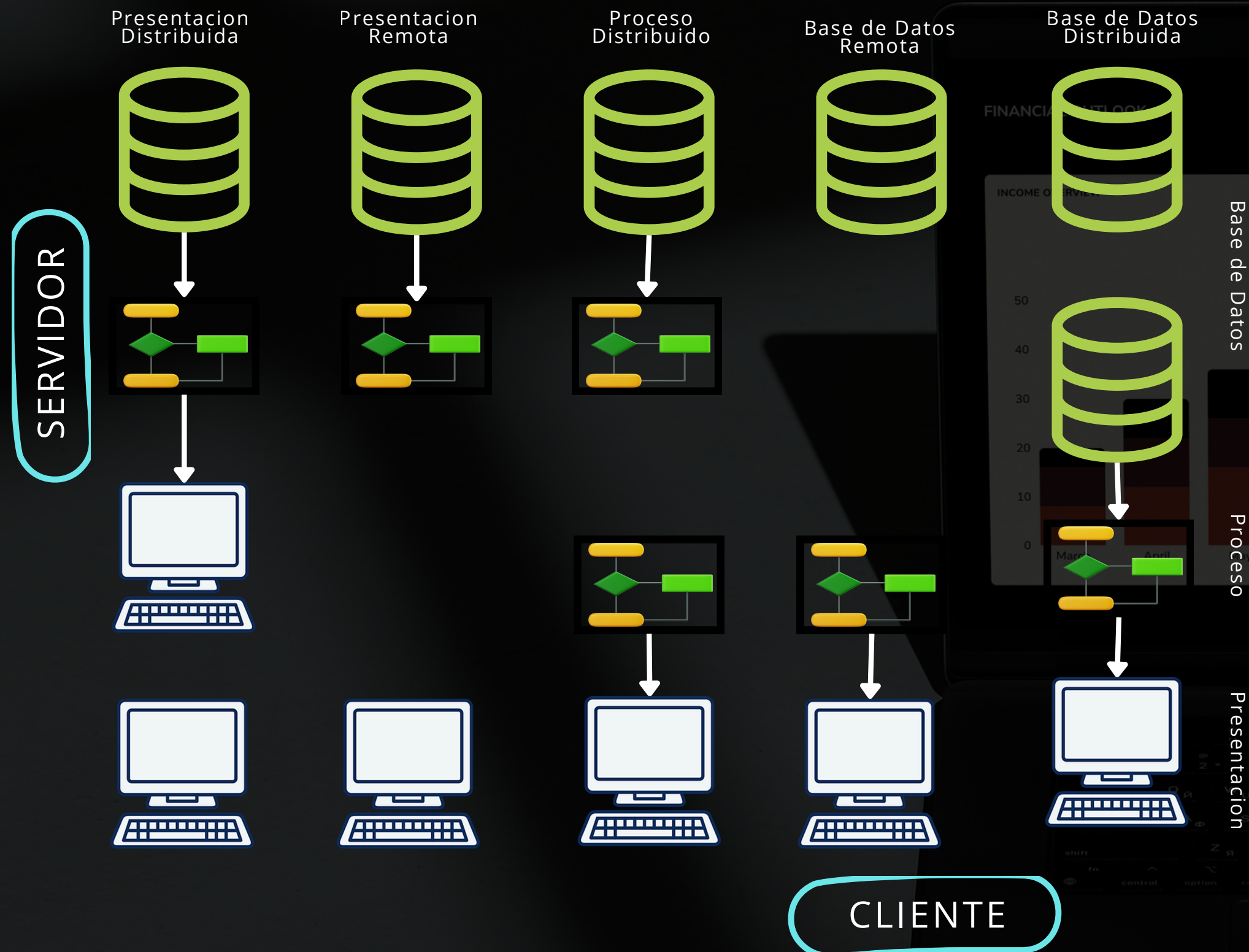
¿QUE ES ?

La arquitectura distribuida es un modelo de software donde los componentes residen en múltiples nodos interconectados en red (servidores, contenedores), funcionando coordinadamente para un objetivo común.

Ventajas

- Escalabilidad horizontal:
 - Alta disponibilidad y tolerancia a fallos
 - Baja latencia geográfica
 - Eficiencia de costos
-
- El "Spaghetti" de red
 - Costos ocultos de infraestructura
 - La pesadilla de la consistencia
 - Dificultad de Observabilidad

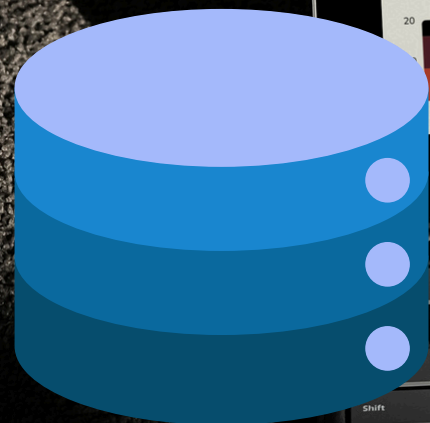
Desventajas



CUADRO COMPARATIVO

PARA LA ARQUITECTURA DE BASE DE DATOS

Arquitecturas	Centralizada	Cliente -Servidor	Distribuida
Velocidad de Consulta (DB)	Instantánea: No hay viaje por red. La lógica y los datos están en el mismo bus.	Dependiente de Red: El "cuello de botella" es el tiempo que tarda el dato en viajar del servidor al cliente	Compleja: Latencia alta si los datos están lejos, pero ultra rápida si se consultan en paralelo.
Carga en la Aplicación	El servidor hace todo el trabajo pesado. El cliente es solo un visor (terminal tonta).	Se reparte: La App procesa la interfaz y el Servidor procesa la lógica y los datos.	La App es inteligente: gestiona múltiples fuentes de datos y sincronización constante.
Consistencia de Datos	Perfecta: Solo hay una copia del dato. No hay versiones contradictorias.	Alta: El servidor es la "fuente de la verdad", aunque el cliente puede tener datos desactualizados (caché).	Un reto: Mantener la misma versión del dato en todos los nodos requiere algoritmos complejos (Consistencia Eventual).
Rendimiento bajo estrés	Se degrada rápido cuando muchos usuarios piden datos al mismo "cerebro"	Se mantiene estable si el servidor es potente, pero sufre con miles de conexiones simultáneas.	Imbatible: El rendimiento escala linealmente. A más demanda, más nodos procesando en paralelo.
Desarrollo (Dev Experience)	Muy Simple: Un solo código, una sola base de datos. Menos errores de integración.	Estándar: Requiere definir APIs/Interfaces claras entre el front y el back.	Muy Difícil: Hay que programar pensando en fallos de red, particionado y réplicas.





GROUND

GRACIAS

POR SU ATENCION



THE INDUSTRY'S HISTORY